

「Git & GitHubを用いたコード管理のいろは」 講義資料

目次

1. Gitとは？
2. GitHubとは？
3. Gitの基本概念
4. Gitのインストールと初期設定
5. 基本的なGitコマンド
6. 実際の作業フロー
7. GitHubを使った共同作業
8. よくある質問（FAQ）
9. GitHubの活用
10. 実践演習

1. Gitとは？

****Git（ギット）****は、バージョン管理システム（Version Control System）の一種です。

なぜGitが必要なのか？

- **変更履歴の管理:** ファイルの変更内容をすべて記録できる
- **過去の状態に戻る:** 過去のバージョンに簡単に戻ることができる
- **複数人での協力:** 同じファイルを複数人で編集しても、変更を統合できる
- **バックアップ:** コードのバックアップとして機能する

Gitの特徴

- **分散型:** ローカルに完全なリポジトリを持てる（インターネットがなくても作業可能）
- **高速:** ローカルで動作するため、操作が速い
- **無料:** オープンソースで無料で使える

2. GitHubとは？

****GitHub（ギットハブ）****は、GitのリポジトリをホスティングするWebサービスです。

「ホスティング」とは？

ホスティングとは、サーバー（インターネット上にあるコンピューター）にデータを保存して、いつでもアクセスできるようにすることです。

具体例で理解する：

- ローカルリポジトリ（自分のパソコン） = 自宅の本棚
- GitHub（ホスティングサービス） = 図書館やクラウドストレージ

自分のパソコンだけでコードを管理すると、パソコンが壊れたら全て失われてしまいます。
GitHubにホスティングすることで：

- **どこからでもアクセス可能:** 別のパソコンからでもコードにアクセスできる
- **バックアップとして機能:** パソコンが壊れてもコードが残る

- **複数人で共有:** チームメンバーとコードを共有できる
- **履歴の保存:** すべての変更履歴がクラウド上に保存される

つまり、GitHubは「Gitリポジトリ専用のクラウドストレージ」のような役割を果たしています。

GitHubの役割

- **リモートリポジトリの保存:** クラウド上にコードを保存できる
- **コード共有:** 他の人とコードを共有できる
- **共同作業:** プルリクエストやイシューを通じて共同開発できる
- **バックアップ:** ローカルのコードが壊れても、GitHubから復元できる

GitとGitHubの違い

Git	GitHub
バージョン管理システム（ツール）	Gitを使うためのWebサービス
ローカルで動作	クラウド上で動作
インストールが必要	Webブラウザでアクセス

3. Gitの基本概念

リポジトリ (Repository)

リポジトリは、プロジェクトの変更履歴を保存する場所です。通常 `.git` フォルダに保存されます。

- ローカルリポジトリ: 自分のパソコン上にあるリポジトリ
- リモートリポジトリ: GitHubなどのサーバー上にあるリポジトリ

コミット (Commit)

コミットは、ファイルの変更内容を保存する操作です。コミットには以下の情報が含まれます：

- 変更したファイルの内容
- コミットメッセージ（何を変更したかの説明）
- 作成者と日時

ブランチ (Branch)

ブランチは、独立した作業領域です。メインのコードに影響を与えずに、新しい機能を開発できます。

- **main/masterブランチ**: 通常、メインのブランチ（本番環境のコード）
- **機能ブランチ**: 新しい機能を開発するためのブランチ

ステージング (Staging)

ステージングは、コミットするファイルを選ぶ操作です。変更したファイルすべてをコミットするのではなく、必要なファイルだけを選びます。

「ステージング」を分かりやすく理解する

ステージングは、買い物で「カゴに入れる」作業に似ています。

具体例：

- スーパーで買い物をするとき、商品を選んでカゴに入れます
- すべての商品をカゴに入れてから、レジで会計します
- 会計する前に、カゴから不要な商品を取り出すこともできます

Gitのステージングも同じです：

- **作業ディレクトリ（ワークツリー）**：スーパーの商品棚（変更したファイルがある場所）
- **ステージング領域（インデックス）**：買い物カゴ（コミットする準備ができたファイルを入れる場所）
- **リポジトリ**：レジで会計が完了した状態（コミットが完了した状態）

ステージングの流れ

```
# 1. ファイルを編集（作業ディレクトリで変更）  
# 例：file1.txt, file2.txt, file3.txt を編集  
  
# 2. 変更内容を確認  
git status  
  
# 3. コミットしたいファイルだけをステージングに追加  
git add file1.txt file2.txt  
# file3.txt はまだステージングに追加していない  
  
# 4. ステージングした内容を確認  
git status  
  
# 5. ステージングしたファイルをコミット  
git commit -m "file1とfile2を更新"
```

ワークツリー、インデックス、リポジトリ

```
作業ディレクトリ（ワークツリー）  
↓ git add  
ステージング領域（インデックス）
```

4. Gitのインストールと初期設定

Gitのインストール

1. [Git公式サイト](#)からダウンロード
2. インストーラーを実行
3. 基本的にはすべて「次へ」でOK

初期設定

インストール後、最初に1回だけ設定します：

```
# ユーザー名の設定
git config --global user.name "あなたの名前"

# メールアドレスの設定
git config --global user.email "your.email@example.com"

# 設定の確認
git config --list
```

5. 基本的なGitコマンド

リポジトリの作成

```
# 新しいリポジトリを作成  
git init
```

```
# 既存のリポジトリをクローン（ダウンロード）  
git clone <リポジトリのURL>
```

ファイルの状態確認

```
# 変更されたファイルを確認  
git status
```

```
# 変更内容の詳細を確認  
git diff
```

ファイルの追加とコミット

```
# 特定のファイルをステージングに追加
git add ファイル名

# すべての変更をステージングに追加
git add .

# ステージングに追加したファイルをコミット
git commit -m "コミットメッセージ"
```

コミット履歴の確認

```
# コミット履歴を表示
git log

# 簡潔な形式で表示
git log --oneline

# グラフ形式で表示
git log --graph --oneline --all
```

過去の状態に戻る

```
# 特定のコミットの内容を確認（ファイルは変更されない）  
git checkout <コミットID>
```

```
# mainブランチに戻る  
git checkout main
```

```
# ファイルの変更を取り消す（まだコミットしていない場合）  
git restore ファイル名
```

```
# ステージングを取り消す  
git restore --staged ファイル名
```

ブランチの操作

```
# ブランチの一覧を表示  
git branch
```

```
# 新しいブランチを作成  
git branch ブランチ名
```

```
# ブランチを切り替える  
git checkout ブランチ名
```

```
# ブランチを作成して切り替える（上記2つのコマンドを同時に）  
git checkout -b ブランチ名
```

```
# ブランチを削除  
git branch -d ブランチ名
```

ブランチの統合（マージ）

```
# mainブランチに切り替え  
git checkout main
```

```
# 他のブランチを統合  
git merge ブランチ名
```

6. 実際の作業フロー

基本的なフロー（1人で作業する場合）

1. リポジトリを作成（初回のみ）

```
git init
```

2. ファイルを作成・編集

3. 変更内容を確認

```
git status
```

4. 変更をステージングに追加

```
git add .
```

5. コミット

```
git commit -m "最初のコミット"
```

6. 繰り返し（2-5を繰り返す）

7. GitHubを使った共同作業

リモートリポジトリの設定

```
# リモートリポジトリを追加
git remote add origin <GitHubのリポジトリURL>

# リモートリポジトリの確認
git remote -v

# mainブランチをリモートにプッシュ（アップロード）
git push -u origin main

# 以後のプッシュ（ブランチ名を省略可能）
git push
```


GitHubでの作業フロー

1. GitHubでリポジトリを作成

1. GitHubにログイン
2. 「New」 ボタンをクリック
3. リポジトリ名を入力
4. 「Create repository」 をクリック

2. ローカルのコードをGitHubにアップロード

```
# リモートリポジトリを追加
git remote add origin https://github.com/ユーザー名/リポジトリ名.git

# コードをプッシュ
git push -u origin main
```

3. GitHubから最新の変更を取得

```
# リモートの変更を取得
```

```
git fetch
```

```
# リモートの変更をマージ
```

```
git pull
```

共同作業の流れ

ブランチを使った作業

```
# 1. 新しい機能ブランチを作成
```

```
git checkout -b feature/new-feature
```

```
# 2. ファイルを編集・コミット
```

```
git add .
```

```
git commit -m "新機能を追加"
```

```
# 3. ブランチをGitHubにプッシュ
```

```
git push -u origin feature/new-feature
```

4. GitHubでプルリクエストを作成
(WebブラウザでGitHubにアクセスして操作)

5. プルリクエストが承認されたらマージ
(GitHub上で操作、またはローカルで)

```
git checkout main  
git pull
```

Issue（イシュー）の活用

Pull Requestは、コードの変更を提案し、レビューを受けるための機能です。

Pull Requestの基本的な流れ

1. 機能ブランチを作成して変更を加える
2. ブランチをGitHubにプッシュ
3. GitHub上で「New Pull Request」をクリック
4. 変更内容を説明して作成
5. レビューを受けて承認されたらマージ

良いPull Requestの書き方

- **明確なタイトル:** 何を変更したかが分かる
- **詳細な説明:** なぜ変更したか、何を変更したかを説明
- **スクリーンショット:** UIの変更がある場合は画像を添付
- **関連Issue:** 関連するIssue番号を記載（例: #123 ）

Issueは、バグ報告や機能要望を管理するための機能です。

Issueの使い方

1. リポジトリの「Issues」タブをクリック
2. 「New Issue」をクリック
3. タイトルと説明を入力
4. ラベルを付けて分類（バグ、機能追加など）
5. 「Submit new issue」で作成

Issueの活用例

- **バグ報告:** 見つけた問題を報告
- **機能要望:** 追加してほしい機能を提案
- **質問:** 使い方について質問
- **タスク管理:** やるべきことをリスト化

8. よくある質問（FAQ）

Q1: `git add` と `git commit` の違いは？

A:

- `git add` : ファイルを「ステージング領域」に追加する（コミットする準備をする）
- `git commit` : ステージング領域の内容を「リポジトリ」に保存する（実際に記録する）

Q2: コミットメッセージは何を書けばいい？

A: 変更内容を簡潔に説明します。例えば：

- "ログイン機能を追加"
- "バグ修正: 計算エラーを修正"
- "READMEファイルを更新"

Q3: 間違えてコミットしてしまった。どうすればいい？

A: まだプッシュしていない場合：

```
# 直前のコミットを取り消す（変更内容は残る）  
git reset --soft HEAD~1
```

```
# 直前のコミットを取り消す（変更内容も消す）  
git reset --hard HEAD~1
```

Q4: `git merge` と `git rebase` の違いは？

A:

- **merge**: 2つのブランチの履歴を統合する（マージコミットが作成される）
- **rebase**: ブランチの履歴を書き直して、一直線の履歴にする

初学者はまず `merge` を覚えましょう。

Q5: `.gitignore` とは？

A: Gitで管理したくないファイルを指定するファイルです。

例： `.gitignore` ファイル

```
# 一時ファイル
*.tmp
*.log

# 環境変数ファイル
.env

# ビルド結果
dist/
build/

# OS固有のファイル
.DS_Store
Thumbs.db
```

Q6: コンフリクト（競合）が発生した。どうすればいい？

A: 同じ部分を複数人が編集した場合に発生します。

1. コンフリクトが発生したファイルを開く
2. 以下のようなマーカーを探す：

```
<<<<<<< HEAD
自分の変更内容
=====
他人の変更内容
>>>>>>> branch-name
```

3. どちらの変更を採用するか、または両方を統合する
4. マーカーを削除
5. `git add` と `git commit` で解決を記録

9. GitHubの活用

GitHub PagesでWebサイトを公開

GitHub Pagesを使うと、リポジトリから直接Webサイトを公開できます。

基本的な使い方

1. GitHubでリポジトリを作成
2. リポジトリの「Settings」→「Pages」に移動
3. 「Source」でブランチを選択（通常は `main` ）
4. 保存すると、 `https://ユーザー名.github.io/リポジトリ名` でアクセス可能

静的サイトの公開例

```
# HTMLファイルを作成
echo "<h1>My Website</h1>" > index.html

# コミットしてプッシュ
git add index.html
git commit -m "Webサイトを追加"
git push
```

READMEファイルの活用

`README.md` ファイルは、リポジトリの説明や使い方を記載する重要なファイルです。

READMEの基本構造

プロジェクト名

概要

プロジェクトの説明

インストール方法

```bash

git clone https://github.com/ユーザー名/リポジトリ名.git

## 使い方

使用方法の説明

## ライセンス

MIT License

# GitHub Actionsの活用

GitHub Actionsは、自動化されたワークフローを実行できる機能です。

## 基本的な使い方

`.github/workflows/` フォルダにYAMLファイルを作成：

```
name: CI

on: [push, pull_request]

jobs:
 test:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v2
 - name: Run tests
 run: npm test
```

## 活用例

- **自動テスト:** コードをプッシュするたびにテストを実行
- **自動デプロイ:** マージされたら自動でデプロイ
- **コードチェック:** コードの品質を自動チェック

## リポジトリの管理

### リポジトリの設定

- **説明:** リポジトリの説明を追加
- **トピック:** 関連するトピックを追加して検索性を向上
- **ライセンス:** 適切なライセンスを選択
- `.gitignore` : 不要なファイルを除外

## ブランチ保護ルール

重要なブランチ（`main` など）を保護する設定：

1. 「Settings」 → 「Branches」 に移動
2. 「Add rule」 をクリック
3. ブランチ名を指定（例: `main` ）
4. 保護ルールを設定：
  - Pull Requestのレビューを必須にする
  - ステータスチェックを必須にする
  - 管理者も含めて保護する



## コントリビューターの追加

1. 「Settings」 → 「Collaborators」 に移動
2. 「Add people」 をクリック
3. ユーザー名またはメールアドレスを入力
4. 権限を設定（Read、Write、Admin）

## コードレビュー

Pull Requestでコードレビューを行う：

- **コメント:** 特定の行にコメントを追加
- **承認:** 変更を承認
- **変更要求:** 修正を要求
- **提案:** コードの改善案を提案

# リリース管理

## リリースの作成

1. 「Releases」 → 「Create a new release」 をクリック
2. タグ名を入力（例: `v1.0.0`）
3. リリースタイトルと説明を入力
4. 「Publish release」 で公開

## セマンティックバージョニング

- メジャーバージョン ( `v1.0.0` ): 互換性のない変更
- マイナーバージョン ( `v1.1.0` ): 後方互換性のある新機能
- パッチバージョン ( `v1.1.1` ): バグ修正

# プロジェクト管理

## Projects機能

GitHub Projectsでタスクを管理：

- **ボード形式:** カンバン形式でタスクを管理
- **Issue連携:** Issueと連携して進捗を管理
- **マイルストーン:** 目標を設定して管理

## Milestones（マイルストーン）

関連するIssueやPull Requestをグループ化：

1. 「Milestones」 → 「New milestone」 をクリック
2. タイトルと説明を入力
3. 期限を設定
4. IssueやPull Requestにマイルストーンを割り当て

# 10. 実践演習

## 演習1: 基本的なGit操作

1. 新しいフォルダを作成
2. `git init` でリポジトリを初期化
3. `README.md` ファイルを作成してコミット
4. ファイルを編集して再度コミット
5. `git log` で履歴を確認

## 演習2: ブランチを使った作業

1. `feature` ブランチを作成
2. ブランチ上でファイルを追加・編集
3. コミット
4. `main` ブランチに戻る

## 演習3: GitHubを使う

1. GitHubでリポジトリを作成
2. ローカルのコードをプッシュ
3. GitHubのWebサイトでコードを確認
4. ローカルで変更して再度プッシュ
5. GitHub上で変更が反映されているか確認

# 11. 参考資料

## 公式ドキュメント

- [Git公式ドキュメント](#)
- [GitHub公式ドキュメント](#)

## 学習リソース

- [Learn Git Branching](#) - ブランチの概念を視覚的に学べる
- [GitHub Skills](#) - GitHub公式の学習コース

## OSS活用事例

- [OSSを活用した「八王子市防災マップ」](#) - GitHub PagesとOSSを活用した防災マップの実例
- [本資料](#)

# よく使うコマンド一覧（チートシート）

# 初期化・クローン

```
git init
git clone <URL>
```

# 状態確認

```
git status
git log
git diff
```

# ファイル操作

```
git add <ファイル>
git add .
git commit -m "メッセージ"
git restore <ファイル>
```

# ブランチ操作

```
git branch
git checkout <ブランチ>
git checkout -b <ブランチ>
git merge <ブランチ>
```

# リモート操作

```
git remote add origin <URL>
git push
git pull
git fetch
```

# まとめ

GitとGitHubは最初は難しく感じますが、基本的な操作を繰り返し練習することで慣れていきます。

## 重要なポイント:

1. 小さく、頻繁にコミットする
2. コミットメッセージは分かりやすく書く
3. ブランチを使って安全に実験する
4. わからないときは `git status` で状態を確認、困ったら `git log` で履歴を確認する

## 次のステップ:

- 実際にプロジェクトでGitを使い始める
- GitHubで他のプロジェクトをフォークしてみる
- プルリクエストを作成してみる